DatingDapp Audit Report

Version 1.0

alva-p.xyz

April 6, 2026

Prepared by: alva-p Lead Auditors: - Pineda Alvaro

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
- Executive Summary
- Findings

Protocol Summary

DatingDapp is a decentralized dating application where users create soulbound profile NFTs (non-transferable ERC721 tokens storing name, age, and profile image on-chain), like each other by staking at least 1 ETH per like, and upon mutual match receive their pooled ETH (minus a 10% protocol fee) into a jointly-controlled 2-of-2 multisig wallet deployed specifically for the matched pair.

The protocol consists of three contracts:

- **SoulboundProfileNFT**: ERC721-based soulbound identity token for user profiles
- **LikeRegistry**: Core matching engine that processes likes, detects mutual matches, collects fees, and deploys multisig wallets
- **MultiSigWallet**: 2-of-2 multisig wallet created per matched pair for shared ETH custody

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Likelihood / Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

- Commit Hash: 878bd34ef6607afe01f280cd5aedf3184fc4ca7b
- Audit Method: Full Pipeline (X-Ray + Deep Audit with 3 specialized agents)

Scope

Subsystem	Key Contracts	nSLOC	Role
Profile	SoulboundProfileNFT.sol	80	Soulbound ERC721 for user identity
Matching	LikeRegistry.sol	61	Like/match engine, fee collection, multisig deployment
Custody	MultiSigWallet.sol	62	2-of-2 multisig wallet for matched pairs

Total: 203 nSLOC

Roles

Actor	Trust Level	Capabilities
Owner	Trusted	Can block any profile (burn NFT), withdraw accumulated fees. All actions instant – no timelock, no multisig on admin.
User	Bounded (must hold profile NFT)	Can mint profile, like other users (>=1 ETH), burn own profile
MultiSig Owner	Bounded (must be matched pair member)	Can submit/approve/execute transactions from their shared multisig

Executive Summary

Threat Model

DatingDapp is classified as a **Social/Incentive** protocol with **Fund Custody** characteristics. The primary attack surfaces are ETH accounting in the like/match flow, the custody model (2-of-2 multisig with no dispute resolution), and access control around profile management. No external oracles, price feeds, or DeFi primitives are used.

Issues found

Severity	Number of issues found
Critical	1
High	2
Medium	3
Low	1
Info	0
Total	7

Findings

Critical

[C-1] `userBalances` never written in `likeUser()` – all deposited ETH permanently locked

`LikeRegistry.likeUser` / `LikeRegistry.matchRewards` – Severity: Critical – Confidence: 95 – [agents: 3]

Description

The `likeUser()` function accepts ETH via `msg.value` (requiring >= 1 ETH) but never credits the sender's balance in the `userBalances` mapping. When a mutual match triggers `matchRewards()`, it reads `userBalances[from]` and `userBalances[to]` – both of which are always 0. This means:

1. `totalRewards = 0`, so `rewards = 0` – the deployed MultiSigWallet receives 0 ETH
2. `matchingFees = 0`, so `totalFees` is never incremented – `withdrawFees()` always reverts
3. All ETH sent via `likeUser()` is permanently locked in the LikeRegistry contract with no recovery mechanism

Attack Path

1. Alice calls `likeUser(Bob)` with `msg.value = 1 ETH` – contract receives 1 ETH, `userBalances[Alice]` remains 0
2. Bob calls `likeUser(Alice)` with `msg.value = 1 ETH` – contract receives 1 ETH, `userBalances[Bob]` remains 0
3. Mutual match detected, `matchRewards(Alice, Bob)` is called
4. `matchUserOne = userBalances[Alice] = 0`, `matchUserTwo = userBalances[Bob] = 0`
5. `totalRewards = 0`, `matchingFees = (0 * 10)/ 100 = 0`, `rewards = 0`
6. New `MultiSigWallet` deployed but receives 0 ETH via `call{value: 0}("")`
7. The 2 ETH sent by Alice and Bob remain permanently locked in `LikeRegistry`
8. Owner calls `withdrawFees()` – reverts with "No fees to withdraw" since `totalFees = 0`

Proof of Concept

```
function testUserBalancesNeverWritten() public {
    // Setup: mint profiles for alice and bob
    vm.prank(alice);
    profileNFT.mintProfile("Alice", 25, "img");
    vm.prank(bob);
```

```
    profileNFT.mintProfile("Bob", 26, "img");

    // Alice likes Bob with 1 ETH
    vm.prank(alice);
    likeRegistry.likeUser{value: 1 ether}(bob);

    // Bob likes Alice with 1 ETH - triggers match
    vm.prank(bob);
    likeRegistry.likeUser{value: 1 ether}(alice);

    // 2 ETH is now locked in LikeRegistry
    assertEq(address(likeRegistry).balance, 2 ether);

    // Owner cannot withdraw fees
    vm.prank(owner);
    vm.expectRevert("No fees to withdraw");
    likeRegistry.withdrawFees();
}
```

Recommended Mitigation

```
  function likeUser(address liked) external payable {
      require(msg.value >= 1 ether, "Must send at least 1 ETH");
      require(!likes[msg.sender][liked], "Already liked");
      require(msg.sender != liked, "Cannot like yourself");
      require(profileNFT.profileToToken(msg.sender) != 0, "Must have a
         profile NFT");
      require(profileNFT.profileToToken(liked) != 0, "Liked user must
         have a profile NFT");

+     userBalances[msg.sender] += msg.value;
+
      likes[msg.sender][liked] = true;
      emit Liked(msg.sender, liked);
```

High

[H-1] No refund mechanism for unmatched likes – ETH permanently locked

`LikeRegistry.likeUser` – Severity: High – Confidence: 90 – [agents: 2]

Description

When a user calls `likeUser()` and the liked user never reciprocates, the deposited ETH (>= 1 ETH) has no withdrawal path. There is no `unlike()`, `refund()`, or `withdraw()` function. The ETH

sits in the contract permanently. Even if the `userBalances` bug (C-1) is fixed, unmatched deposits remain irrecoverable.

Attack Path

1. Alice calls `likeUser(Bob)` with 1 ETH – `userBalances[Alice] += 1 ETH` (assuming C-1 fix)
2. Bob never calls `likeUser(Alice)` – no mutual match ever triggers
3. Alice has no function to call to recover her 1 ETH
4. `withdrawFees()` only withdraws protocol fees, not user balances
5. `matchRewards()` only triggers on mutual match
6. Alice's 1 ETH is permanently locked

Recommended Mitigation

```
+ function withdrawBalance() external {
+     uint256 balance = userBalances[msg.sender];
+     require(balance > 0, "No balance to withdraw");
+     userBalances[msg.sender] = 0;
+     (bool success,) = payable(msg.sender).call{value: balance}("");
+     require(success, "Transfer failed");
+ }
```

[H-2] Aggregated `userBalances` drain on first match – deposits for other likes stolen

`LikeRegistry.matchRewards` – Severity: High – Confidence: 75

Description

Even with the `userBalances` fix applied, the balance is tracked per-user rather than per-pair. When a user likes multiple people, all their deposits accumulate into a single `userBalances[user]`. When any one match triggers, `matchRewards()` drains the user's **entire** balance to that single MultiSigWallet, leaving zero for subsequent matches.

This means deposits intended for other potential matches are redirected to whichever match triggers first.

Attack Path

1. Alice likes Bob (1 ETH) – `userBalances[Alice] = 1 ETH`
2. Alice likes Carol (1 ETH) – `userBalances[Alice] = 2 ETH`
3. Bob likes Alice – mutual match triggers `matchRewards(Alice, Bob)`

4. `matchUserOne = userBalances[Alice] = 2 ETH` – Alice's **entire** balance drained
5. Alice-Bob MultiSig receives (2 + 1) * 0.9 = 2.7 ETH instead of expected 1.8 ETH
6. Carol likes Alice later – `userBalances[Alice] = 0` – Alice-Carol MultiSig receives only Carol's deposit

---

Medium

[M-1] `blockProfile()` trivially bypassed – blocked users can immediately re-mint

`SoulboundProfileNFT.blockProfile`/`SoulboundProfileNFT.mintProfile` – Severity: Medium – Confidence: 85 – [agents: 2]

Description

The `blockProfile()` function burns the user's NFT and deletes `profileToToken[user]`, setting it back to 0. However, `mintProfile()` only checks `require(profileToToken[msg.sender] == 0)` – the exact state that `blockProfile` creates. A blocked user can immediately call `mintProfile()` again to re-enter the system, completely nullifying the admin's blocking action.

Attack Path

1. Owner calls `blockProfile(maliciousUser)` – NFT burned, `profileToToken[maliciousUser] = 0`
2. In the very next block, `maliciousUser` calls `mintProfile("newName", 25, "img")`
3. `require(profileToToken[msg.sender] == 0)` passes – user gets a new token ID
4. User is fully active again, the block had zero lasting effect

Recommended Mitigation

```
+ mapping(address => bool) public blocked;

  function blockProfile(address blockAddress) external onlyOwner {
      uint256 tokenId = profileToToken[blockAddress];
      require(tokenId != 0, "No profile found");

+     blocked[blockAddress] = true;
      _burn(tokenId);
      delete profileToToken[blockAddress];
      delete _profiles[tokenId];

      emit ProfileBurned(blockAddress, tokenId);
```

```
  }

  function mintProfile(string memory name, uint8 age, string memory profileImage) external {
      require(profileToToken[msg.sender] == 0, "Profile already exists");
+     require(!blocked[msg.sender], "Address is blocked");
```

[M-2] Stale likes persist across profile burns – matches trigger with wrong profile context

`LikeRegistry.likeUser` – Severity: Medium – Confidence: 80

Description

When a user burns their profile (via `burnProfile()`) or gets blocked (via `blockProfile()`), their entries in the `likes` and `matches` mappings in `LikeRegistry` are not cleared. If the user re-mints a profile, old likes from a previous profile session can trigger matches with the new profile, creating inconsistent state.

Attack Path

1. Alice mints profile (tokenId=1) and likes Bob
2. Alice burns profile – `profileToToken[Alice] = 0`, but `likes[Alice][Bob] = true` persists
3. Alice mints new profile (tokenId=2) with different data
4. Bob likes Alice – `likes[Bob][Alice]` check passes, `likes[Alice][Bob]` is still true from step 1
5. Match triggers based on a like from Alice's old, deleted profile

---

[M-3] Griefing via spam likes and MultiSig blocking

`LikeRegistry.likeUser` – Severity: Medium – Confidence: 75

Description

Any user can force themselves into another user's matches list by liking them and waiting for reciprocation. Once matched, the attacker co-controls the MultiSigWallet as owner1 or owner2 and can permanently block all transactions by refusing to approve. There is no mechanism to reject matches,

remove a match partner, or resolve disputes in the MultiSig. The 2-of-2 multisig has no timeout, no fallback, and no escape hatch.

Low

[L-1] MultiSigWallet lacks reentrancy guard – cross-transaction atomic drain possible

`MultiSigWallet.executeTransaction` – Severity: Low – Confidence: 70 – [agents: 2]

Description

`executeTransaction()` sets `txn.executed = true` before the external `call{value}`, which prevents same-transaction reentrancy. However, if multiple transactions are pre-approved, a malicious recipient contract can re-enter `executeTransaction()` with a **different** `txId` during the callback, executing multiple transactions atomically in a single call frame. While both owners must have approved all transactions, the atomic execution prevents the second owner from reacting or revoking between executions.

Leads

Vulnerability trails with concrete code smells where the full exploit path could not be completed. These are not false positives – they are high-signal leads for manual review.

- **Unbounded matches array growth** – `LikeRegistry.likeUser` – Code smells: `matches[user].push()` grows without bound; `getMatches()` returns full array. If a user accumulates many matches, off-chain consumers calling `getMatches()` will hit gas limits.
- **Untracked ETH via receive()** – `LikeRegistry.receive` – Code smells: `receive() external payable {}` with empty body accepts arbitrary ETH without updating any accounting state. Force-fed or directly-sent ETH is permanently locked with no recovery path.
- **No transaction cancellation in MultiSig** – `MultiSigWallet.submitTransaction` – Code smells: No mechanism to cancel a submitted transaction. Transactions with values exceeding the wallet balance will revert on execution but remain permanently in the pending state.

- **Missing constructor address validation** – `LikeRegistry.constructor` – Code smells: `_profileNFT` parameter not validated as a contract address. Deployment with wrong address would require full redeployment since there is no setter.

Architecture

For the full X-Ray readiness report, see `x-ray/x-ray.md`. For the complete entry point map, see x-ray/entry-points.md. For the architecture diagram, see x-ray/architecture.svg.

> This review was performed by an AI-assisted audit pipeline. AI analysis cannot guarantee the complete absence of vulnerabilities. Manual security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.